LABORATORIO S11

Dos bases más y una caché

Sistemas Distribuidos 2027-1 · Grupo 2 · 7 de noviembre de 2026

PostgreSQL conserva la verdad del dinero. MongoDB guarda la bitácora de avisos con documentos según el canal. Redis mantiene una copia temporal del saldo para mostrarlo más rápido; nunca participa en la autorización de cargos.

Dato	Almacén	Semántica en esta práctica
Cuenta y saldo	PostgreSQL	Verdad; transacción local y versión
Avisos enviados	MongoDB	Bitácora documental, append-only e idempotente
Saldo de consulta	Redis	Copia derivada; TTL de 30 s e invalidación al cambiar

Identificadores del experimento

Cuenta de Ana: 002180000000000001. Beto: 002180000000000002. Carla: 002180000000000003. Saldos iniciales: $1,000, $500 y $300. Las transferencias pequeñas y los ensayos de la bitácora modificaron los saldos durante la práctica; el estado final quedó Ana $973, Beto $522 y Carla $305.

Entorno y ajuste de ejecución

Se levantó el compose del laboratorio con tres instancias de cuenta. En este Docker Desktop, MongoDB 8.0 se detenía por incompatibilidad con el kernel Linux que usa Docker Desktop; se fijó la imagen en MongoDB 7.0.45 y los servicios arrancaron correctamente. La nota oficial de MongoDB documenta el problema de TCMalloc en kernels recientes.

El punto de partida

Diez GET consecutivos de Ana produjeron diez lecturas de PostgreSQL. En ese momento Mongo tenía cero avisos. Esto confirma que antes de activar la caché cada consulta de saldo llegaba a la base relacional.



Captura 1. Medición inicial: diez consultas y diez lecturas de PostgreSQL; Mongo vacío.

01 / Caché compartida de saldo

La caché se aplica en la consulta de presentación. La operación que autoriza un cargo sigue consultando PostgreSQL.

En ConfiguracionCache se habilitó el caching y se declaró la caché cuentas, TTL de 30 segundos, sin valores nulos, JSON de CuentaVista con el ObjectMapper de Spring y configuración transaccional. ConsultaDeSaldo.consultar usa @Cacheable con la CLABE como clave. En un miss ejecuta la lectura de PostgreSQL y Redis recibe la respuesta; en un hit Spring responde desde Redis y el método no vuelve a leer la base.

En cargar y abonar se usa @CacheEvict. Se borra la copia, no se calcula y escribe un nuevo valor en Redis. La configuración transaction-aware difiere la invalidación hasta que la transacción se confirma. CuentaService.consultar, que autoriza el cargo, sigue leyendo PostgreSQL directamente. Así el valor derivado jamás decide si existe saldo suficiente.

Evidencia de caché compartida

Se hicieron seis GET distribuidos entre las tres instancias. El encabezado X-Instancia cambió entre caf4677b52d7, 7fe652990c11 y 36a455b04eea, pero las respuestas conservaron la misma leidaEn y el saldo. Redis tenía JSON legible para cuentas::002180000000000001, con TTL 30. El resultado demuestra que las réplicas comparten Redis.



Captura 2. Las tres instancias responden con la misma entrada compartida; se ve el JSON y su TTL.

Invalidación

Antes de una transferencia de $1, EXISTS para la clave de Ana era 1. La transferencia respondió 201 COMPLETADA y, justo después, EXISTS fue 0. El GET siguiente volvió a leer la fuente y rellenó Redis con el saldo actualizado. El saldo presentado cambió de $980 a $979 en la prueba aislada de invalidación.

02 / Mediciones

Se repitieron tres corridas de 100 GET para latencia con caché y sin caché.

Modo	Corrida	Promedio (ms)	Mediana (ms)	p95 (ms)	Lecturas PostgreSQL
Con caché	1	8.46	6.27	18.50	1 por 100 GET
Con caché	2	5.33	5.19	6.46	1 por 100 GET
Con caché	3	5.12	4.98	6.55	1 por 100 GET
Sin caché	1	14.67	8.01	11.25	100
Sin caché	2	10.00	8.50	16.58	100
Sin caché	3	8.07	7.56	10.94	100

Escenario A: 100 consultas consecutivas, 99 hits y 1 miss, hit ratio 99%. En las tres corridas con caché la lectura a PostgreSQL fue una; sin caché fueron 100 por corrida. En este equipo local, Redis redujo trabajo de PostgreSQL y normalmente mejoró la latencia, aunque hay variación y se traslapan algunos percentiles: la ganancia más sólida aquí fue capacidad de lectura de la base, no una promesa de menor latencia en cada solicitud.

Escenario B: 100 consultas intercaladas con una transferencia de $1 cada diez consultas. Redis reportó 100 hits y 20 misses: hit ratio agregado de 83.33%. Hubo 20 lecturas a PostgreSQL en 100 GET. Los misses incluyen las búsquedas fallidas para invalidar la clave de destino cuando aún no estaba cacheada, por eso los contadores agregados de Redis no equivalen al número de misses de GET por sí solo. Cada transferencia invalida el saldo que cambió y fuerza una recarga en el siguiente GET.

La caché permite servir muchas lecturas repetidas con una sola lectura de PostgreSQL. El precio es aceptar que el saldo mostrado pueda atrasarse hasta el TTL si una escritura evita la invalidación normal.

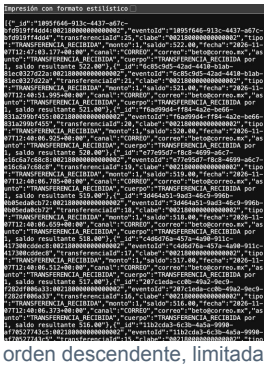
03 / Avisos en MongoDB

La bitácora tiene documentos distintos por canal y se consulta por CLABE, del más nuevo al más viejo.

El consumidor guarda cada aviso en MongoDB y no hace una escritura paralela a PostgreSQL. Los campos comunes incluyen eventoId, transferenciaId, clabe, tipo, monto, saldo, fecha y canal. Los valores monetarios se guardan como Decimal128; la fecha del documento proviene del evento.

Canal	Campos específicos observados
SMS · Ana	telefono, texto
CORREO · Beto	correo, asunto, cuerpo
PUSH · Carla	token, titulo, cuerpo

El endpoint GET /api/notificaciones/{clabe} devuelve hasta diez documentos ordenados por fecha descendente. En la captura se ve la bandeja de Beto. Para soportar la consulta se creó el índice compuesto clabe_fecha con clabe ascendente y fecha descendente. El explain con índice fue IXSCAN y examinó 10 claves/documentos; al forzar $natural el plan fue COLLSCAN + SORT y examinó 44 documentos en esa medición.



Captura 3. Respuesta real del endpoint de Beto: avisos en orden descendente, limitada a diez.



Captura 4. Formas de los tres canales, tipo Decimal128 e inspección del plan de consulta.

04 / Evento duplicado

La entrega de RabbitMQ es al menos una vez; el consumidor debe poder procesar el mismo evento de nuevo.

Se publicó dos veces el evento fijo 11111111-2222-4333-8444-555555555555. Se esperaban dos documentos, uno por CLABE destinataria del evento, y Mongo conservó exactamente 2, no 4. La clave determinista _id = eventoId + ":" + clabe hace que un segundo intento de insertar el aviso de la misma persona choque con DuplicateKeyException. La excepción se trata como “ya estaba” y el proceso continúa con el siguiente destinatario.

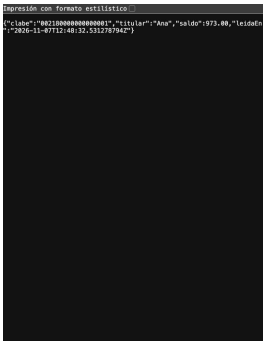
El arreglo es idempotencia en el almacenamiento: mismo evento y destinatario producen la misma identidad documental. El registro del consumidor confirmó los intentos repetidos como existentes.



Captura 5. Conteo del evento repetido, logs de idempotencia y evidencia del saldo stale.

Dato viejo y límite de la caché

Se calentó la entrada de Ana en Redis con saldo $973 y después se actualizó PostgreSQL directamente a $5, omitiendo deliberadamente el @CacheEvict. La respuesta cacheada continuó mostrando $973 con leidaEn=2026-11-07T12:48:32.531278794Z. Tras expirar la entrada, la respuesta pasó a $5 y leidaEn=2026-11-07T12:49:30.033211001Z. La observación confirma una ventana de dato viejo limitada por el TTL configurado de 30 segundos; la duración observada depende de cuánto TTL quedaba al hacer el UPDATE.



Captura 6. Cliente todavía ve la copia previa de $973, con su leidaEn original.

05 / Dato viejo: expiración y autorización

La presentación puede estar atrasada; el cargo se decide con la verdad de PostgreSQL.



Captura 7. Tras la expiración, GET vuelve a PostgreSQL y muestra $5 con una leidaEn nueva.

Mientras Redis aún presentaba el saldo anterior, se intentó transferir $100. La API respondió 422, Saldo insuficiente. La autorización consultó PostgreSQL, donde el saldo real era $5. El resultado demuestra por qué una caché de presentación no se debe usar para autorizar movimientos: habría permitido decidir usando un valor obsoleto. Después del experimento se restauró en PostgreSQL el saldo de Ana a $973 y se eliminó la entrada cacheada.

Preguntas de cierre

¿Por qué Mongo para avisos y no para saldo? Los avisos tienen esquemas por canal y se leen como documentos completos por CLABE; Mongo permite guardar esas formas. El saldo debe conservar consistencia transaccional con cargos, abonos y movimientos. Por eso su verdad permanece en PostgreSQL.

¿Qué hit ratio y qué se ganó? A: 99% (99/100); B: 83.33% en contadores Redis (100/120). PostgreSQL bajó de 100 a 1 lectura en A, y leyó 20 veces en B debido a cambios. En este equipo el beneficio más claro fue capacidad; la latencia mejoró en general pero tuvo variación.

¿Qué pasaría si autorización usara la caché? Podría aprobar un cargo con saldo viejo. La prueba mostró que Redis podía mostrar $973 cuando PostgreSQL ya tenía $5; la autorización correcta respondió 422.

¿Por qué DEL y después del commit? Borrar evita escribir en caché un saldo que luego no se confirme. Después del commit la base ya tiene el nuevo estado. La próxima consulta recarga el valor autoritativo. La configuración transaction-aware ordena el efecto de caché tras confirmar.

06 / Reto: Redis caído

Se verificó que una falla de una copia derivada no derribe el servicio de cuenta.

Antes de ajustar el manejo de errores, apagar Redis hizo que GET y transferencia respondieran 500. Se configuró un CacheErrorHandler que registra e ignora fallas de get/put/evict/clear; la operación de negocio sigue usando PostgreSQL. Con Redis apagado, GET respondió 200 desde Postgres y una transferencia de $1 respondió 201 COMPLETADA. Los logs dejaron advertencias para GET, PUT y DEL. Al reactivar Redis, el servicio continuó y la caché volvió a poblarse.

La razón para tolerar errores de caché incluye el instante del DEL: con transactionAware puede ocurrir después de que el cargo ya se confirmó. Si un error de DEL propagara un 500, la saga podría interpretar como fallida una operación de dinero que sí se aplicó y reintentarla. El handler evita comunicar esa falla derivada como fallo de negocio; el TTL acota el tiempo de una copia que no pudo invalidarse.



Captura 8. Prueba con Redis detenido: error inicial y recuperación del GET/transferencia tras el handler.

Alcance y estado de entrega

Se entrega el código de cuenta y notificacion sin directorios target, el compose ajustado a MongoDB 7.0.45 y esta bitácora. No se realizó checkout a la rama de entregas, commit ni push. Las capturas están integradas en este PDF.

Estado final observado: Ana $973, Beto $522, Carla $305; Mongo conserva avisos de los ensayos, Redis volvió a estar disponible. Las cifras corresponden a las ejecuciones registradas el 7 de noviembre de 2026.

Nota de versión MongoDB: https://www.mongodb.com/docs/manual/release-notes/8.0/